

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: July 1, 2026
Semester: 1st Semester	Session: Spring 2026
Lab/Project/Assignment #: 5	CLOs to be covered: CLO2
Lab Title: Functions	Teacher Name: Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO2	Design and implement modular programs using functions, recursion, and reusable code constructs.					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 5

Lab Goals / Objectives:

- Understand the concept and importance of functions in Python
- Learn all Python built-in functions with practical examples
- Understand function definition, parameters, and return values
- Master different types of function arguments
- Learn about recursive functions and their applications
- Understand the scope and lifetime of variables in functions
- Apply functions to solve real-world problems

Equipment Required:

- Computer System with Python 3.x installed
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

Theory:

1. What is a Function?

A function is a reusable block of code that performs a specific task. Functions help break down large programs into smaller, manageable pieces. They promote code reuse and make programs more organized and easier to maintain.

2. Types of Functions in Python

Python provides two types of functions:

- Built-in Functions: Pre-defined functions available in Python (e.g., print(), input(), len(), type())
- User-defined Functions: Functions created by the programmer using the def keyword

3. Function Definition Syntax

Functions are defined using the def keyword followed by the function name and parentheses containing optional parameters:

```
def function_name(parameters):  
    """Docstring (optional)"""  
    # Function body  
    return value # Optional
```

4. Built-in Functions in Python

4.1 print() Function

The print() function outputs the specified message to the console. It can take multiple arguments and supports various formatting options through sep and end parameters.

```
print("Hello, World!")  
print("Name:", "Ali", sep="- ")
```

4.2 input() Function

The `input()` function reads a line from input, converts it to a string, and returns it. It optionally takes a prompt message as an argument.

```
name = input("Enter your name: ")
```

4.3 type() Function

The `type()` function returns the data type of the specified object. It is useful for debugging and type checking.

```
type(10)      # <class 'int'>
type(3.14)    # <class 'float'>
type("Hello") # <class 'str'>
```

4.4 int(), float(), str() Functions

These functions convert values to integer, floating-point, and string types respectively:

```
int("10")     # Converts string to integer
float("3.14")  # Converts string to float
str(100)       # Converts integer to string
```

4.5 len() Function

The `len()` function returns the number of items in an object such as a string, list, tuple, or dictionary:

```
len("Hello")   # Returns 5
len([1, 2, 3]) # Returns 3
```

4.6 max() and min() Functions

The `max()` function returns the largest item in an iterable or the largest of two or more arguments. The `min()` function returns the smallest.

```
max(10, 25, 7) # Returns 25  
min(10, 25, 7) # Returns 7
```

4.7 `sum()` Function

The `sum()` function adds all items in an iterable and returns the total:

```
sum([1, 2, 3, 4]) # Returns 10
```

4.8 `range()` Function

The `range()` function generates a sequence of numbers. It is commonly used with for loops:

```
range(5)          # 0, 1, 2, 3, 4  
range(1, 6)       # 1, 2, 3, 4, 5  
range(0, 10, 2)   # 0, 2, 4, 6, 8
```

4.9 `abs()` Function

The `abs()` function returns the absolute value of a number:

```
abs(-10) # Returns 10  
abs(3.14) # Returns 3.14
```

4.10 `round()` Function

The `round()` function rounds a number to a given precision in decimal digits:

```
round(3.14159, 2) # Returns 3.14
```

4.11 `pow()` Function

The `pow()` function returns the value of `x` to the power of `y` (x^y). It is equivalent to the `**` operator:

```
pow(2, 3)    # Returns 8
```

4.12 sorted() Function

The `sorted()` function returns a new sorted list from the elements of any iterable:

```
sorted([3, 1, 4, 1, 5]) # Returns [1, 1, 3, 4, 5]
```

5. Function Arguments

5.1 Positional Arguments

Arguments passed in the order they are defined. The position determines which parameter receives the value.

5.2 Keyword Arguments

Arguments passed with explicit parameter names, allowing them to be in any order.

5.3 Default Arguments

Parameters that have default values. If the caller does not provide a value, the default is used.

5.4 Variable-Length Arguments (*args)

Allows a function to accept any number of positional arguments. The arguments are collected into a tuple.

6. The return Statement

The return statement exits a function and passes a value back to the caller. A function can return multiple values as a tuple.

7. Docstrings

Docstrings are string literals that appear as the first statement in a function. They document what the function does and can be accessed via the `__doc__` attribute.

```
def greet():  
    """This function prints a greeting message."""  
    print("Hello!")
```

Lab Work:

Task 1: Built-in Functions Demonstration

Demonstrate the use of various Python built-in functions with practical examples.

Code Summary:

This task demonstrates essential built-in functions including `type()` for checking data types, `len()` for finding length, `max()` and `min()` for comparisons, and `abs()` for absolute value.

Key Code Lines:

```
# Demonstration of Built-in Functions  
print("Type of 10:", type(10))  
print("Length of 'Hello':", len("Hello"))  
print("Max of 10, 25:", max(10, 25))  
print("Min of 10, 25:", min(10, 25))  
print("Absolute of -15:", abs(-15))  
print("Power 2^3:", pow(2, 3))
```

Output:



```
Run 5 x  
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 4\5.py"  
Type of 10: <class 'int'>  
Length of 'Hello': 5  
Max of 10, 25: 25  
Min of 10, 25: 10  
Absolute of -15: 15  
Power 2^3: 8  
  
Process finished with exit code 0
```

Task 2: Type Casting Functions

Demonstrate type casting using `int()`, `float()`, and `str()` functions.

Code Summary:

This task shows how to convert between different data types using built-in casting functions, which is essential for handling user input and performing calculations.

Key Code Lines:

```
# Type Casting Demonstration
x = "100"
y = int(x)
print("String to int:", y, type(y))

a = 3.14
b = str(a)
print("Float to string:", b, type(b))

c = 10
d = float(c)
print("Int to float:", d, type(d))
```

Output:

A screenshot of a Python terminal window. The window has a title bar with 'Run' and a tab labeled '5'. The terminal shows the execution of a Python script. The output is as follows:

```
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 4\5.py"
String to int: 100 <class 'int'>
Float to string: 3.14 <class 'str'>
Int to float: 10.0 <class 'float'>

Process finished with exit code 0
```

Task 3: Greet Function

Define and call a simple function that prints a welcome message.

Code Summary:

This task demonstrates the basic structure of a user-defined function: definition using `def`, function body with statements, and calling the function by name.

Key Code Lines:

```
# Simple Greet Function
def greet():
    print("Welcome to Python Programming")

greet()
greet()
```

Output:



```
Run 5 x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 4\5.py"
Welcome to Python Programming
Welcome to Python Programming
Process finished with exit code 0
```

Task 4: Function with Parameters and Return

Define a function that takes two numbers as parameters and returns their sum.

Code Summary:

This task demonstrates passing parameters to a function and returning a computed result using the `return` statement.

Key Code Lines:

```
# Function with Parameters and Return
def addNumbers(a, b):
    Total = a + b
    return Total
```

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
print("Sum =", addNumbers(num1, num2))
```

Output:



```
Run 5 x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 4\5.py"
Enter first number: 77
Enter second number: 33
Sum = 110
Process finished with exit code 0
```

Task 5: Function with Default Argument

Define a function `power(base, exponent=2)` that calculates power with a default exponent of 2.

Code Summary:

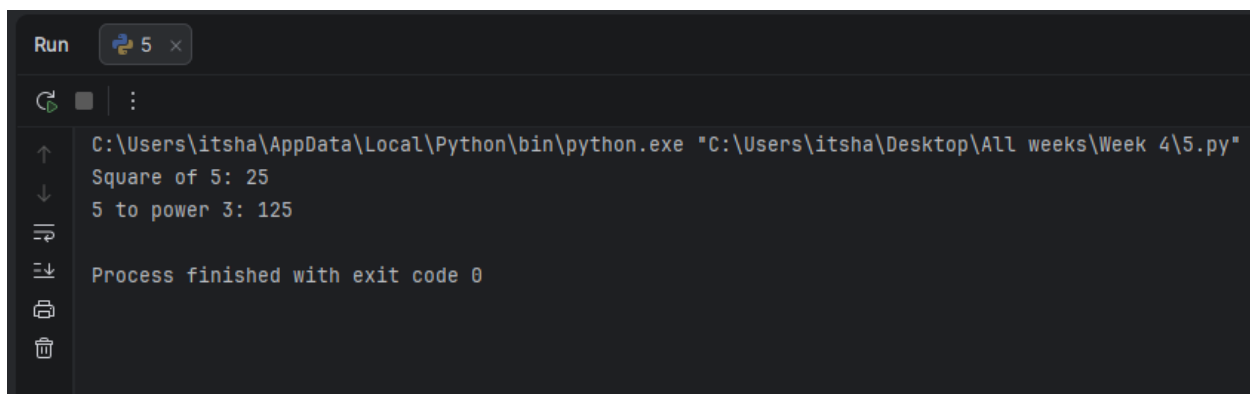
This task demonstrates default parameter values. When the second argument is omitted, the default value is used.

Key Code Lines:

```
# Function with Default Argument
def power(base, exponent=2):
    return base ** exponent

print("Square of 5:", power(5))
print("5 to power 3:", power(5, 3))
```

Output:



```
Run 5 x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 4\5.py"
Square of 5: 25
5 to power 3: 125
Process finished with exit code 0
```

Conclusions:

- Successfully understood the importance and working of functions in Python
- Learned and demonstrated all major Python built-in functions
- Mastered function definition with def keyword and various argument types
- Understood positional, keyword, default, and variable-length arguments
- Learned to use the return statement for passing values back to callers
- Understood the concept of docstrings for function documentation
- Applied functions to write cleaner, reusable, and modular code